

Identifying strategies in *Dominion* using playtrace clustering

Anthony Owen

Independent Researcher, anthonyowen1@gmail.com

Abstract—We demonstrate the use of playtraces and playtrace clustering to identify strategies and card synergies in deck building card games, using *Dominion* as an example. We analyse two different types of playtraces generated from both online human play and a variety of AI agents. Firstly we examine playtraces defined by tracking card counts by round in a player’s deck, and secondly utilising N-Grams generated from player actions. We investigate playtrace clustering using both the L_k -norm and Jensen-Shannon distance measures, in-conjunction with K -Means, K -Medoids and DBSCAN algorithms. We show that distinct playtrace clusters can arise from different strategies, and that playtraces and cluster centroids provide a means to identify both longer term strategies and also short-term tactics involving particular card combinations. Additionally, we use a restricted play framework to increase the variation in strategies and tactics explored by the AI agent. We conclude by highlighting how the game agnostic N-Gram based approach might be used to explore strategy spaces in tabletop games more broadly.

Index Terms—Clustering, board games, deck building card games, playtraces, Monte Carlo Tree Search.

I. INTRODUCTION

Deck building card games (DBGs) are a genre of modern card games that began with *Dominion* (Rio Grande Games, 2008). These card games are many and varied, with each game tending to have its own unique set of cards. Examples include *Ascension* (Stone Blade Entertainment, 2010), *Star Realms* (Wise Wizard Games, 2014), *Star Wars: A deck building game* (Fantasy Flight Games, 2023) and many others.

In DBGs players typically have cards in their hand, a draw pile and a discard pile. Each player takes turns to play cards from their hand and add new cards to their discard pile by purchasing from a central common supply. Each played card generates some in-game resource or activates some effect (or both), that can move a player towards a prescribed victory condition. A player’s hand is replenished by draws from the draw pile, if this becomes exhausted, the discard pile is shuffled and becomes the new draw pile. In this way, a player’s deck evolves as the game progresses, and effectively the player is designing an engine through card choices to achieve a specific goal.

Elements of deck building card games have also been fused with other gameplay elements in both board games and video games. For example in the board game *Clank!* (Renegade Game Studios, 2016), cards in a player’s deck are used to both move a player token around a board and fight monsters. In the video game *Slay the Spire* (Mega Crit, 2017), battles involve drawing and playing cards in a similar manner to the DBG mechanic described above. At the end of each battle,

the player must choose a card from a selection of three options to add to their deck, and hence their deck evolves through time.

Collectible card games (CCGs) are close cousins to DBGs. Classic examples of CCGs include *Magic: The Gathering* (Wizards of the West Coast, 1993), *Pokémon Trading Card Game* (Media Factory, 1996) and *Hearthstone* (Blizzard Entertainment, 2014). The main difference compared to DBGs, is that a player’s deck is constructed as part of a meta-game and not during gameplay. However the author expects that the N-Gram based playtraces discussed here, could also be used in this genre to explore card synergies and strategies.

A fundamental challenge of DBGs is that the impact of each card can depend on which other cards exist in a player’s deck or hand. This creates a combinatorial problem, for example a DBG can easily have 100 distinct cards, assuming a rough deck size of 20 cards, this leads to approximately 5×10^{20} possible decks. As a consequence, identifying all possible strategies and card synergies can be a challenging task. This in turn makes game design and game balancing time consuming and difficult.

The main contribution of this paper is to demonstrate that using playtraces and playtrace clustering from human and AI simulated play of deck building card games, can provide a means to identify available strategies and tactics. We use *Dominion* as a case study analysing two different types of playtraces and the impact on clustering of using a variety of clustering algorithms and distance measures between playtraces.

II. RELATED WORK

The use of playtraces has a long history in the field of videogames. Examples include the analysis of playtraces to identify player styles [6] and to understand player behaviour [7]. Additionally [8] investigates a variety of dissimilarity metrics and clustering algorithms to understand player activity in some example levels from stealth and platform games.

Construction of playtraces for arbitrary games was discussed in [9]. Here playtraces are developed from in-game actions and a distance metric based on the edit distance is used to group similar playtraces together, in order to visualise strategies. However the examples given relate only to videogames.

The use of playtraces and playtrace clustering within the context of board and card games has been much more limited. To the authors knowledge, only [10] has specifically investigated the clustering of playtraces within a modern tabletop

game. They investigate playtraces generated from human play of *Dominion* obtained from an online game server. Clusters are visually identified using t-distributed stochastic neighbour embedding, with the resulting visualisations qualitatively examined for player strategies.

The use of AI based simulated playtesting of games has also predominantly been applied to analysing game balance, as opposed to generation and analysis of playtraces. Furthermore, the literature related to automated game balancing of DBCGs is limited to three examples, two of which utilise *Dominion* as a test case. These three examples consist firstly of [11], which utilises genetic algorithms to assess game balance in *Dominion* with respect to the availability of specific game mechanics (represented by a common set of cards that players can obtain during game play). Secondly, in [12] Monte Carlo Tree Search (MCTS) is used to run repeated game sessions evaluating the influence of particular cards and pairs of cards on the outcome. Finally, [13] discusses an auto-battler card game developed for the express purpose of AI research, and proposes a number of metrics that could be used to assess meta-game health in deck building card games.

This paper contributes to the existing literature by demonstrating how both AI simulated game sessions and human play of a DBCG can be used to generate playtraces, which when clustered can aid in strategy identification and card synergies.

III. BACKGROUND

A. *Dominion*

Dominion is a deck building card game published by Rio Grande Game and designed by Donald Vaccarino. We give a brief description of how play proceeds below, a full description can be found in the rule book¹.

There are seven base cards that are common for any game of *Dominion*. Four are victory point (VP) cards, Curse (-1VP), Estate (1VP), Duchy (3VP) and Province (6VP). The remaining three base cards are the treasure cards Copper, Silver and Gold, worth 1, 2 and 3 treasures respectively. Each card in the game has an associated cost which can be paid for by using these treasure cards. The VP cards have a limited supply dependent on the number of players.

Next we have a Kingdom card set, which is comprised of ten card piles, each containing ten copies of a single card. The ten card types used in the Kingdom are chosen by the players and different choices give rise to different mechanics and strategies during the game.

Each player has a draw pile, a discard pile and receives a starting deck consisting of seven Copper cards and three Estate cards. At the start of each round each player draws five cards into their hand from their draw pile. Players take turns in order and each turn is split into three phases: an Action phase, a Buy phase and a Clean phase.

In the Action phase, the player may play a single action card (a card that is not a VP card or a treasure card, and is typically a Kingdom card) from their hand, and resolve the effect of that card. Some action cards provide a '+1 Action',

which then allows the player to play another action card from their hand, and so on. Once a player runs out of actions they move onto the Buy phase.

In the Buy phase, all treasure generated in the action phase can be used to purchase a new card from either the Kingdom or base cards. Each player can buy one card per Buy phase, or more if they increased their limit during the Action phase.

Finally in the Clean up phase, all the cards played and bought by the player are added to their discard pile, and a new set of five cards is drawn in preparation for the next round. If the draw pile has been exhausted, the discard pile is shuffled and used to form a new draw pile.

The game finishes if either the Province pile is empty, or any three or more of the Kingdom card piles are empty. Each player then sums up the total amount of victory points they have in their deck, the player with the most victory points wins.

B. *Tabletop Games Framework*

The implementation of *Dominion* including the Monte Carlo Tree Search (MCTS) algorithm (see [23] for an introduction to MCTS) and tournament functionality used in this paper are provided by the Tabletop Games Framework (TAG) [14]. The purpose of TAG is to promote research into the uses of AI in modern tabletop games. It provides a common API for games and game playing agents, allowing for a comparison of AI methods across a broad spectrum of tabletop games. It is used to support cross board game analyses and also to explore reinforcement learning in board games [15].

Specifically TAG contains an implementation of *Dominion*, that has all the game features and is easily expandable to incorporate additional AI agents and Kingdom card sets. Additionally, the game engine for *Dominion* in TAG is well supported and has been used in other research.

IV. METHODS

A. *Clustering algorithms and visualisation*

We will focus on three main clustering approaches. Firstly we use *K*-Means [18], which is both relatively simple and computationally fast. However it requires the selection of the number of clusters, *K*, upfront as a hyperparameter, and assumes clusters are spherically distributed. Additionally, *K*-Means requires a Euclidean distance measure to guarantee convergence. This can be a disadvantage, since it suffers from the 'curse of dimensionality', i.e. in higher dimensions, the discriminatory power of the Euclidean distance deteriorates [19].

Secondly we will use *K*-Medoids [20], a variant of *K*-Means. Here cluster centroids are not computed as averages but instead data points from the input data set are chosen. This is particularly relevant in our case, as an average across all playtraces may be misleading in suggesting card combinations that are not played together in any individual playtrace. The *K*-Medoids algorithm can be used with non-Euclidean distance measures.

Finally we also employ Density-Based Spatial Clustering of Applications with Noise (DBSCAN). DBSCAN has two

¹www.riograndegames.com/wp-content/uploads/2016/09/Dominion2nd.pdf

TABLE I
KINGDOM CARD SET - SIZE DISTORTION

Name	Effect Summary	Cost
Chapel	Trash up to 4 cards from your hand	3
Workshop	Gain a card costing up to 4	3
Throne Room	Play an action card in hand twice	4
Bureaucrat	Gain a Silver, opponents redraw VP cards	4
Gardens	Worth 1VP for every 10 cards in player's deck	4
Bandit	Gain a Gold, opponents trash 1 treasure card	5
Festival	+2 Actions, +1 Buy, +2 treasure	5
Witch	+2 Cards, opponents gain a Curse	5
Sentry	+1 Card, +1 Action, trash 2 cards	5
Artisan	Gain a card costing up to 5	6

hyperparameters, m and ε , that define the minimum number of neighbouring points for a point to be considered a core point, and the size of that neighbourhood respectively. Identification of core points is used to determine clusters, as discussed in [21]. The main benefits of this approach include identifying non-spherical clusters, assigning data points as noise as opposed to forcing a cluster assignment and supporting non-Euclidean distance measures.

In this case the concept of a cluster centroid can be harder to define. For simplicity, we will define a cluster centroid as the nearest playtrace to the mean playtrace for a cluster. This may or may not be representative of the cluster as a whole, depending on how close the cluster is to being spherically distributed.

We visualise the resultant clusters using metric multi-dimensional scaling (MDS) [22]. MDS takes as an input a distance matrix, and reconstructs a set of coordinates in a lower dimensional space that reproduces as closely as possible the original distance matrix. This mapping to a lower dimensional space allows visualisation of our clusters.

B. Finite State Machine agents

We use two finite state machines (FSM) that implement two well known *Dominion* strategies, when simulating our game sessions. These strategies are specific to the ‘size distortion’ (SD) Kingdom card set described in table I (see the *Dominion strategy wiki*² for a full description of card effects). This Kingdom card set is recommended in the base game of *Dominion*.

Our first FSM agent plays a Big Money with Gardens strategy (BMWG) with the SD set of cards. Big Money is a base case strategy in *Dominion*, and focuses on buying treasure cards that in turn allow the purchase of Province cards for victory points. Big Money with Gardens is a variant of the Big Money Strategy, adding the purchasing of Gardens to generate additional victory points.

Our second FSM agent implements the Double Witch strategy. This strategy aims at using the Witch card to flood an opponent’s deck with Curse cards. This not only reduces their victory point score, but also reduces the probability of the opponent drawing useful cards³.

We will use FSM agents primarily as a base case, and a means of demonstrating how distinct strategies can result in distinct clusters within our playtrace data set.

²<https://wiki.dominionstrategy.com/>

³Code for both our FSM agents can be found at https://github.com/owenant/TabletopGames_ForDominionPlayTraces

TABLE II
AVERAGE WIN RATES OVER 100 GAMES WITH THE SD KINGDOM CARD SET (HIGHEST WIN RATES IN BOLD).

	MCTS	BMWG	DW
MCTS	53%	83%	40%
BMWG	17%	50%	1%
DW	60%	99%	50%

C. Monte Carlo Tree Search agent

In addition to our FSM agents, we also use an MCTS based agent. We use a single MCTS agent that combines both Information Set MCTS (ISMCTS) [24] and Multi-Tree MCTS [25]. In ISMCTS, each node in the tree is defined by a set of actions from the root node, as opposed to being a specific game state at each node. Additionally, hidden information is re-sampled at the root node for each simulation loop. In multi-tree MCTS, a tree is constructed independently for each player

ISMCTS helps with the hidden information aspects of *Dominion* and multi-tree MCTS trades off greater search depth at the expense of reacting to opponent’s moves. Due to the indirect nature of the interaction between players in *Dominion*, this trade-off is expected to be positive.

Our MCTS agent will also use the following features,

- 1) UCT-tuned [26] as our tree policy in the selection phase (with $K = 1$).
- 2) The simulation play-outs are performed to a length of 30 actions, with a playout policy determined by a simple score based heuristic (actions are chosen to maximise the player’s score for the next turn).
- 3) The playouts also use MAST [27], with the MAST gamma value set to 0.5, and with an explore epsilon set to 0.1
- 4) A time budget of 500 milliseconds will be allotted for each move.

Note that our choice of heuristic is inline with empirical evidence on the performance of MCTS in *Dominion* given in [28]. However, we found that a roll-out length of 30 combined with MAST gives a superior performance to a longer roll-out with a random play-out policy (57% win rate).

We compare win rates between our MCTS agent and FSM agents using the SD Kingdom card set. The average win rates are given in table II, with each win rate given as the win rate for the AI agent in the row versus the AI agent in the column. Note that the DW agent is the strongest, however this strategy is specific to the SD card set, and the MCTS agent is capable of playing any Kingdom card set.

V. PLAYTRACE DEFINITIONS AND DISTANCE MEASURES

A. Card count playtraces and L_k -norm

A card count playtrace is defined by the number of each type of card in a player’s deck at the end of each round. Firstly, we denote a deck vector for player p in game n at the end of round t by $\vec{d}_n^{p,t}$. This is a vector of length M , the number of distinct card types available in the supply. Entries are denoted by $d_{n,i}^{p,t} \in \mathbb{Z}_0^+$, the positive number of cards of type i in a player’s deck (across draw, hand, discard and played card piles) at the end of round t . Then a card count playtrace, $\vec{T}_n^p$, is defined by

$$\vec{T}_n^p = (\vec{d}_n^{p,1}, \vec{d}_n^{p,2}, \dots, \vec{d}_n^{p,R}), \quad (1)$$

where R is the number of rounds.

As our playtrace is a vector of integer vectors, we can equivalently treat our playtrace as a single integer vector of length MR . It follows that we can define a distance between two playtraces utilising the L_k -norm,

$$\text{dist}^k(\vec{T}_{n_1}^{p_1}, \vec{T}_{n_2}^{p_2}) = \left(\sum_{t=1}^R \sum_{i=1}^M (d_{n_1,i}^{p_1,t} - d_{n_2,i}^{p_2,t})^k \right)^{\frac{1}{k}}. \quad (2)$$

Note that for k equal to two we have the usual Euclidean distance measure. Also, we can normalise a playtrace vector $\vec{T}_n^p$ by diving each entry by $\text{dist}^k(\vec{T}_n^p, \vec{0})$, where $\vec{0}$ is a vector of zeros with the same length as $\vec{T}_n^p$.

Not all games of *Dominion* will last the same number of rounds, so R will be defined as the largest number of rounds across all games played in our game sessions. Deck vectors will then be frozen when a game ends and copied, so that the count for each card type will remain constant from the final game round up to round R . This ensures that each playtrace has the same length and we can apply our distance measure.

In our case, given that R can be of the order of 50, our playtraces will have roughly 850 dimensions (M is seventeen in *Dominion*). Fractional L_k -norms, i.e. with k less than one are suggested by [19] to have better discriminatory power in higher dimensions. Therefore, we will also explore the impact of varying k in the formation of our playtrace clusters.

B. N-Grams and Jensen-Shannon distance

An alternative approach to using the game state space is to use the action space. We define a string, S_n^p , that defines the list of actions taken by player p in game n . For example,

PlayWitch BuySilver PlayWorkshop GainGardens PlayChapel TrashCopper
BuyProvince ...

Next we map this string to a playtrace, $S_n^p(N)$, by converting it into a multiset of N-Grams. An N-Gram is a concept from Natural Language Processing [29] and is a sequence of N words or in our case, actions. For our simple example and in the case of bi-grams, each action is paired with neighbouring actions to generate a multiset of action pairs,

{(PlayWitch, BuySilver), (BuySilver, PlayWorkshop), (PlayWorkshop, GainGardens), (GainGardens, PlayChapel), (PlayChapel, TrashCopper) ...}

A uni-gram based playtrace is a multiset containing individual actions, whereas for higher order N-Grams our playtrace consists of a multiset containing all observed sequences of N actions taken in order.

We employ two simplifying assumptions to make our playtraces easier to interpret. Firstly, we introduce an aggregated action called 'End Current Phase' (ECP). This action is used whenever a player ends their turn or only plays Treasure cards and does not purchase or play cards from the Kingdom card set. Additionally, we do not record actions that are in response to an opponent's actions. For example, if an opponent plays a Militia card, we do not record which two cards are discarded

by the player. These simplifications allow us to focus more on the strategy being played by an individual player and helps with interpretability of the playtraces.

To quantify the distance between N-Gram based playtraces we use the Jensen-Shannon measure, which measures the degree of dissimilarity between two probability distributions. For a given N-Gram playtrace, $S_n^p(N)$, we map to a corresponding discrete probability distribution, $SP_n^p(N)$, with a domain given by the set $V(N)$, of all N-grams observed across playtraces in our game session. Then for each N-Gram $G_i \in V(N)$, we count the number of times we observe that N-Gram in $S_n^p(N)$. If an N-Gram is not observed in a given playtrace the count is set to zero, we then normalise our counts to compute probabilities.

We are now in a position to calculate our Jensen-Shannon distance measure between two playtraces, $S_{n_1}^{p_1}(N)$ and $S_{n_2}^{p_2}(N)$. To simplify notation we define $p = SP_{n_1}^{p_1}(N)$, $q = SP_{n_2}^{p_2}(N)$ and $m = \frac{1}{2}(p + q)$ which is the average of our two probability distributions. Additionally p_i and q_i denote the probabilities associated with N-Gram i from p and q respectively. Then we have,

$$\text{dist}^{JS}(S_{n_1}^{p_1}(N), S_{n_2}^{p_2}(N)) = \sqrt{\frac{1}{2}(KL(p, m) + KL(q, m))}, \quad (3)$$

where,

$$KL(p, q) = \sum_{i=1}^{|V(N)|} f(p_i, q_i). \quad (4)$$

Here $f(p_i, q_i) = p_i \log\left(\frac{p_i}{q_i}\right)$, if both p_i and q_i are non-zero, otherwise it is zero. Note that the Jensen-Shannon distance measure has the benefit of accommodating playtraces of variable length.

VI. EXPERIMENTS

We perform a number of experiments using both FSM and MCTS agents competing in round robin tournaments of 500 games, with each bot switching between being first and second player after 250 games. These tournaments are run using two different Kingdom card sets in TAG. Data on player actions and the game state space is recorded at the end of each round, and then processed to form our card count and N-Gram based playtraces.

In order to determine the optimal hyperparameters for our various clustering methods, we perform a grid search maximising the average of silhouette coefficients across clusters. The silhouette coefficient for a single data point is a measure of how close that data point is to all other data points within the cluster to which it belongs, versus the distance to all the other clusters (see [30]). A silhouette coefficient close to one means that a data point is close to all the other points in the cluster to which it belongs, and also far apart from points in the neighbouring cluster. Conversely if the silhouette coefficient is close to minus one, it is more similar to points in a neighbouring cluster.

For the grid search for K -Means and K -Medoids we let K vary between 2 and 5 clusters. For DBSCAN, we vary m from 5 to 250 in steps of size 10, and ε from 0.01 to

TABLE III
SILHOUETTE AVERAGES (SA) WITH A TOURNAMENT OF 500 GAMES FOR
BMWG VERSUS DW USING THE SD CARD SET

Playtrace	Algorithm	Norm	SA	Hyperparams
Card count	K -Means	L_2	0.59	$K = 2$
Card count	K -Medoids	L_2	0.59	$K = 2$
Card count	K -Medoids	$L_{0.5}$	0.80	$K = 2$
Card count	DBSCAN	L_2	0.58	$m = 5, \varepsilon = 0.16$
Card count	DBSCAN	$L_{0.5}$	0.74	$m = 5, \varepsilon = 0.14$
Uni-gram	K -Medoids	JS	0.67	$K = 2$
Bi-gram	K -Medoids	JS	0.52	$K = 2$
Tri-gram	K -Medoids	JS	0.38	$K = 2$
Uni-gram	DBSCAN	JS	0.67	$m = 5, \varepsilon = 0.11$
Bi-gram	DBSCAN	JS	0.52	$m = 5, \varepsilon = 0.26$
Tri-gram	DBSCAN	JS	0.38	$m = 95, \varepsilon = 0.46$

0.995 in steps of 0.005. The maximum value for m is based on our data sample size (we will run 500 games for MCTS). Additionally, we normalise our card count playtraces so that the maximum distance between any two playtraces is two. We set a maximum ε as half that maximum value.

A. FSM agents with SD Kingdom card set

We start by establishing a baseline set of results using the two FSM agents corresponding to BMWG and DW strategies. We run a tournament consisting of 500 games using the SD kingdom card set. Optimal silhouette averages from our grid search (along with the corresponding hyperparameters) for our various playtrace types and clustering algorithms are shown in table III.

We see that in the card count case the $L_{0.5}$ norm gives the highest silhouette average, inline with our expectations that in a high dimensional space the discriminatory power of the Euclidean norm is reduced. We also see that the uni-gram result is slightly higher than the L_2 card count case, however we see a deterioration in silhouette average as we move from uni- to tri-grams. For uni-, bi- and tri-grams there are 78, 870 and 3694 distinct N-Grams observed respectively in our tournament. As the order of the N-Grams increases and due to the relatively short length of a *Dominion* game, it becomes progressively more unlikely that playtraces will share common sequences of actions, which leads to an average overall increase in distance between playtraces and a diffusion of clusters.

Another observation is that our DBSCAN and K -Medoids results are very close, suggesting that our clusters are roughly spherical. The DBSCAN and K -Medoids MDS plots are virtually identical in this case, and therefore we only show some example DBSCAN MDS plots in figure 1.

The corresponding DBSCAN centroids for the card count case with $L_{0.5}$ -norm are shown in figure 2. Cluster zero clearly shows a DW strategy, since the Witch card count reaches a maximum of two, and there is a focus on Silver and Gold to purchase Province and Duchy VP cards. Cluster one shows a BMWG strategy, which can be observed by the focus on treasure cards and the purchase of Gardens cards for additional VPs. Note that the purchase of Province cards is somewhat limited, this is driven by the influx of Curse cards from the DW opponent, weakening the strength of the agent's deck.

Examining the centroids in the bi-gram case in figure 3, we again see our two strategies reflected in the clusters. Cluster

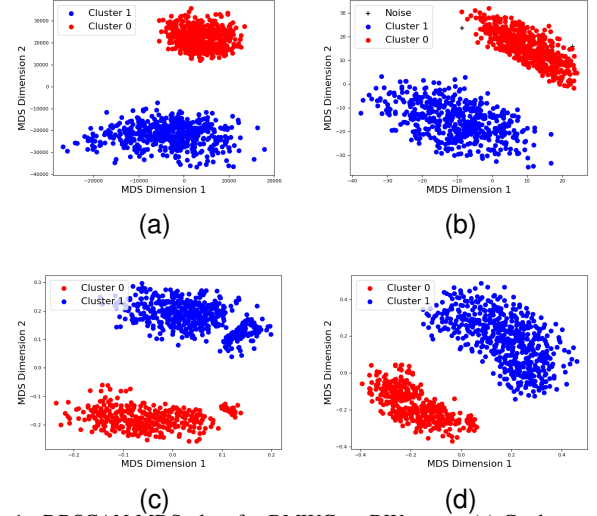


Fig. 1. DBSCAN MDS plots for BMWG vs DW agents (a) Card count with $L_{0.5}$ (b) Card count with L_2 (c) Uni-grams (d) Bi-grams.

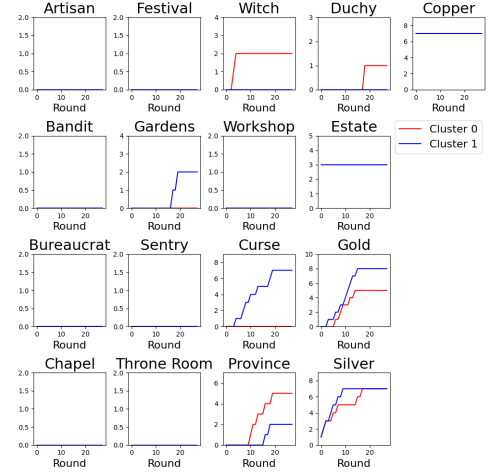


Fig. 2. DBSCAN centroids for BMWG vs DW agents with card count playtraces and $L_{0.5}$ -norm.

one reflects a BMWG strategy and has a high frequency of bi-grams containing a buy Gold or buy Silver action, reflecting it's focus on treasure cards. Additionally, we see purchasing of Garden cards for additional VPs. Cluster zero follows a DW strategy and has a number of bi-grams containing either a play or buy Witch action. The remaining bi-grams for cluster zero do not contain any other Kingdom cards, but solely focus on treasure cards and VP cards.

This demonstrates that clustering of card count and N-Gram playtraces is capable of separating out different strategies, at least in this simplified case with two players playing separate and clearly defined strategies.

B. Monte Carlo Tree Search with SD Kingdom card set

Next we move onto the case of MCTS, where we do not know the strategy being played upfront. We again use the SD Kingdom card set and run a tournament consisting of 500 games. We remove outlier games with a length greater than 55 rounds or with a score higher than 100, which account for 1% of our playtraces. Silhouette averages are shown in table IV.

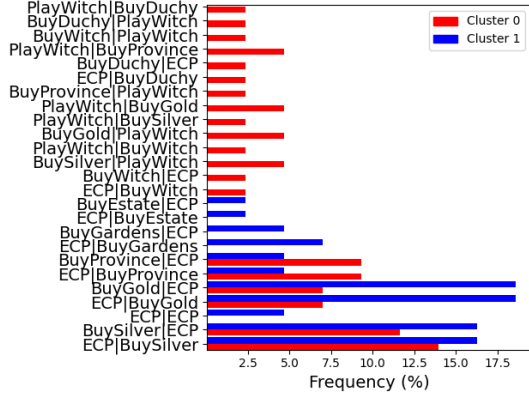


Fig. 3. DBSCAN centroids for BMWG vs DW agents for bi-grams.

TABLE IV
SILHOUETTE AVERAGES WITH A TOURNAMENT OF 500 GAMES BETWEEN
MCTS AGENTS USING THE SD CARD SET

Playtrace	Algorithm	Norm	SA	Hyperparams
Card count	K -Means	L_2	0.27	$K = 2$
Card count	K -Medoids	L_2	0.27	$K = 2$
Card count	K -Medoids	$L_{0.5}$	0.35	$K = 2$
Card count	DBSCAN	L_2	0.35	$m = 115, \epsilon = 0.315$
Card count	DBSCAN	$L_{0.5}$	0.4	$m = 95, \epsilon = 0.37$
Uni-gram	K -Medoids	JS	0.26	$K = 2$
Bi-gram	K -Medoids	JS	0.15	$K = 2$
Tri-gram	K -Medoids	JS	0.07	$K = 2$
Uni-gram	DBSCAN	JS	0.35	$m = 135, \epsilon = 0.26$
Bi-gram	DBSCAN	JS	0.21	$m = 185, \epsilon = 0.44$
Tri-gram	DBSCAN	JS	0.11	$m = 95, \epsilon = 0.6$

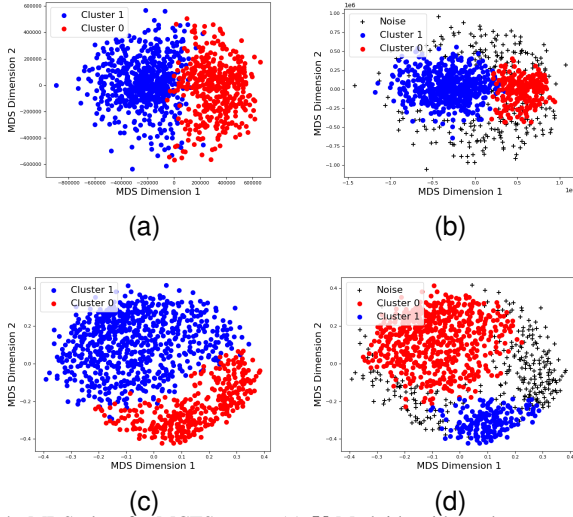


Fig. 4. MDS plots for MCTS agents (a) K -Medoids with card count and $L_{0.5}$ (b) DBSCAN with card count and $L_{0.5}$ (c) K -Medoids with Uni-grams (d) DBSCAN with Uni-grams.

We see much lower silhouette averages as compared to the FSM case, since both players are simulated using the same MCTS agent as opposed to two different bots with distinct strategies. Except for the bi- and tri-gram cases, the silhouette averages are strong enough to indicate the presence of clusters, and the silhouette averages for K -Means and K -Medoids are maximised with K equal to two. The presence of two clusters can also be confirmed through visualisation using the MDS plots in figure 4. We see two clusters that are clearer in the DBSCAN case where data points can be assigned as noise as opposed to being forced into a specific cluster.

We again examine our cluster centroids shown in figures 5

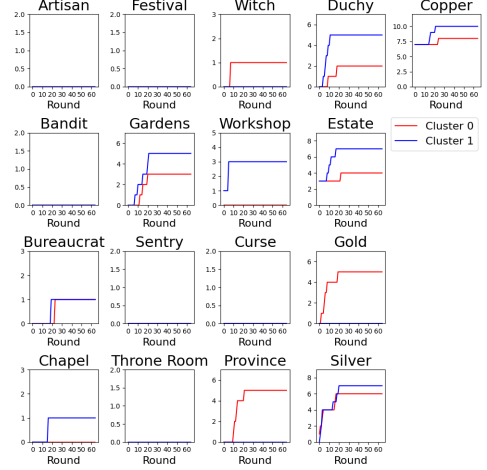


Fig. 5. DBSCAN centroids for MCTS agents with card count playtraces and $L_{0.5}$ -norm.

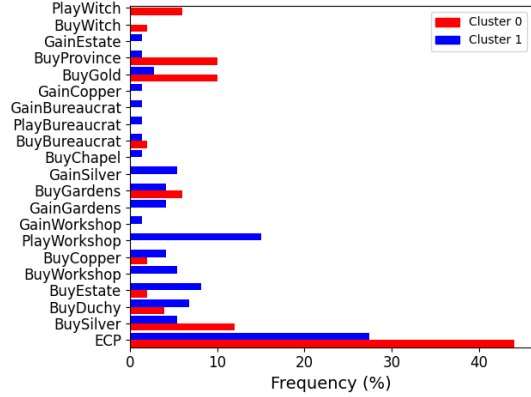


Fig. 6. DBSCAN centroids for MCTS agents for Uni-grams.

and 6. The cluster centroids for the DBSCAN case are similar to the equivalent K -Medoids case.

The two clusters show distinct strategies. The cluster one centroid displays a well know 'Gardens/Workshop' combination focus. This is shown in the card count case by these being the main Kingdom cards purchased, and in the uni-gram case by the relatively high frequency of play Workshop, and also buy and gain Gardens action. Garden cards cost 4 and are worth 1VP for each 10 cards in the player's deck. The Workshop card allows a player to pick up any card with a cost of 4 or less for free. In this case, the strategy is to use the Workshop cards to gain Garden cards (in addition to purchasing them directly), generating victory points.

The cluster zero centroid has a strong focus on treasure cards, particularly Gold and also purchasing Province cards and some Garden cards for victory points. This is a Big Money with Gardens strategy. We also see some use of the Witch card. Note that we do not include gain Curse in our N-Grams actions as it is a response to another player's action, and those actions are ignored to simplify our N-Grams. The lack of curse cards in the centroids for the card count case, seems to suggest that the Witch card is not played enough by our MCTS agent to generate enough curse cards that they appear in the cluster centroids.

Additionally, we note that there is limited use of the Chapel

TABLE V
KINGDOM CARD SET - FIRST GAME 1ST EDITION

Name	Effect Summary	Cost
Cellar	+1 Action, discard x cards then draw x cards	2
Moat	+2 Cards, reveal to protect from attack	2
Village	+1 Card, +2 Actions	3
Woodcutter	+1 Buy, +2 treasure	3
Workshop	Gain a card costing up to 4	3
Smithy	+3 Cards	4
Remodel	Trash a card and gain a card worth 2 more	4
Militia	+2 treasure, opponents discard down to 3 cards	4
Market	+1 Card, +1 Action, +1 Buy, +1 treasure	5
Mine	Trash a treasure, gain a treasure worth 3 more	5

TABLE VI
SILHOUETTE AVERAGES FOR MCTS AGENTS AND HUMAN PLAY USING
THE FG1E CARD SET

Playtrace	Algorithm	Norm	SA-MCTS	SA-Human
Card count	K -Means	L_2	0.17	0.15
Card count	K -Medoids	L_2	0.14	0.10
Card count	K -Medoids	$L_{0.5}$	0.16	0.12
Card count	DBSCAN	L_2	0.34	N/A
Card count	DBSCAN	$L_{0.5}$	0.57	N/A
Uni-gram	K -Medoids	JS	0.30	0.08
Bi-gram	K -Medoids	JS	0.06	0.10
Tri-gram	K -Medoids	JS	0.02	0.04
Uni-gram	DBSCAN	JS	0.20	0.39
Bi-gram	DBSCAN	JS	0.24	N/A
Tri-gram	DBSCAN	JS	N/A	N/A

card. This is commonly known to be a strong card in *Dominion* that allows a player to trash or remove weaker cards from their deck, thereby increasing the overall strength of their deck. The MCTS agent does not tend to make use of this card, this is most likely due to the impact of trashing requiring a relatively deep exploration of the game tree for the benefits to become apparent. These examples show that not all available strategies are captured by our MCTS agent.

C. Comparison of human and MCTS generated playtraces with the FG1E Kingdom card set

Using MCTS generated playtraces allows us to generate a large amount of data quickly, however human playtesting is essential. In this section we compare playtraces generated by MCTS to those generated by human play.

In order to do this, we examine player logs collected by dominion.isotropic.org between October 2010 and March 2013. The most popular Kingdom card set is the First Game 1st Edition (FG1E) set, a summary of this Kingdom card set is given in table V. Restricting ourselves to two player games we find 149 relevant log files. The actions of each player for each round is parsed and then converted into card count and N-Gram based playtraces. Then in-conjunction with an MCTS tournament of 500 games played with the FG1E kingdom card set, we can compare clustering between AI and human play. We start by examining the silhouette averages in table VI.

In the cases where we have N/A the algorithm is assigning all playtraces to a single cluster, and hence we can not compute the silhouette average. The non-zero DBSCAN silhouette averages are also somewhat misleading, as we generally have one large cluster with a few remaining playtraces assigned to a second cluster. Examples are shown in figure 7. The only case where we see clustering is in the MCTS uni-gram case for K -Medoids, shown in figure 7d.

Figure 8 shows the the cluster centroids in the K -Medoids uni-gram case. The centroids show similar strategies. Cluster

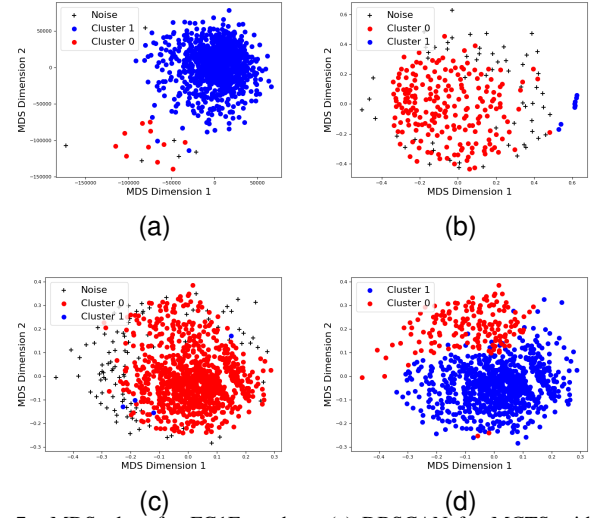


Fig. 7. MDS plots for FG1E card set (a) DBSCAN for MCTS with card counts and $L_{0.5}$ -norm (b) DBSCAN for human players with Uni-grams (c) DBSCAN for MCTS agent with Uni-grams (d) K -Medoids for MCTS with uni-grams.

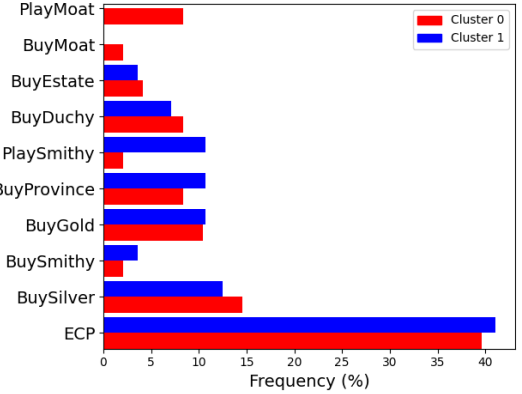


Fig. 8. Cluster centroids for K -Medoids uni-gram clustering of MCTS generated playtraces for FG1E card set.

one is showing a Smithy plus Big Money strategy, which focuses on collecting treasure cards such as Silver and Gold in order to buy Provinces, and then speeds up this process by using Smithy to draw extra treasure cards. Cluster zero is similar, except makes more use of the Moat card as opposed to the Smithy, which allows for a smaller number of new card draws but is also cheaper.

Given the lack of observable clustering for this Kingdom card set, we compute an average playtrace across all available playtraces, to give us a sense of the average play of both our MCTS and human players. In figure 9 we have the average card count playtraces for both human and MCTS agents, and in figure 10 the corresponding uni-gram case (note we have removed uni-grams with a frequency of less than 1.5% for clarity).

Here we see more evidence that the MCTS agent is generally playing a Smithy plus Big Money strategy. This is clear from the average card count playtrace where we see the main action card being purchased by the MCTS agent is Smithy. Additionally, the uni-gram playtrace shows a focus on playing Smithy, buying Gold and Silver and also the Province and Duchy VP cards.

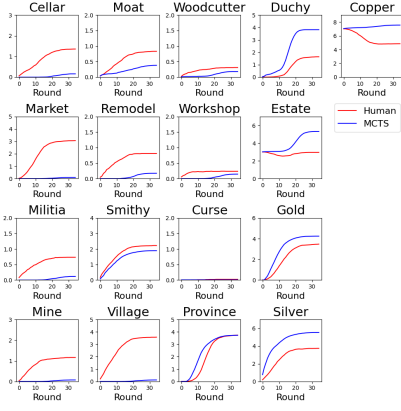


Fig. 9. Mean card count playtraces with $L_{0.5}$ -norm for human players and MCTS agents for FG1E card set.

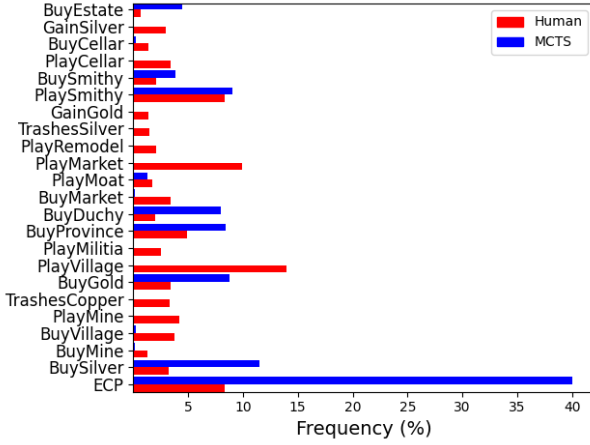


Fig. 10. Mean uni-gram probability distribution for human players and MCTS agents for FG1E card set.

For human play, we see more varied play as compared to the MCTS case. In particular, we see much more use of Village and Market cards, in-conjunction with Smithy. This reflects a more advanced Smithy/Village engine based strategy. In this strategy the aim is to cycle through the entire deck each round, with the Village providing additional cards and actions that allow a subsequent play of a Smithy. Ideally the treasure in a player's deck should then be enough to buy a Province or Duchy card each time. Typically a Market card is also added to provide additional buy actions so that more than one victory card can be bought at a time.

Another difference between our MCTS and human based playtraces is the latter's use of the Mine card. This allows a player to trash Copper cards, which are weak. This increases a player's probability of drawing higher value treasure cards and is a common technique in *Dominion* for strengthening a deck. This tactic was not obviously exhibited in the MCTS case.

D. Using restricted play to increase variation in MCTS playstyle

As we have seen in previous sections, our MCTS agent does not use all available strategies and the generated playtraces provide only a restricted view on card synergies. To partially alleviate the impact of this problem, we can use the restricted

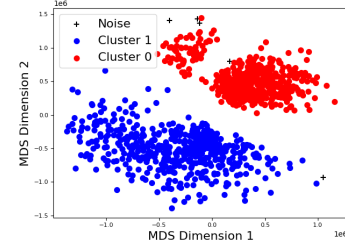


Fig. 11. MDS DBSCAN plot for cardcount playtraces with $L_{0.5}$ -norm and MCTS restricted play with SD Kingdom card set.

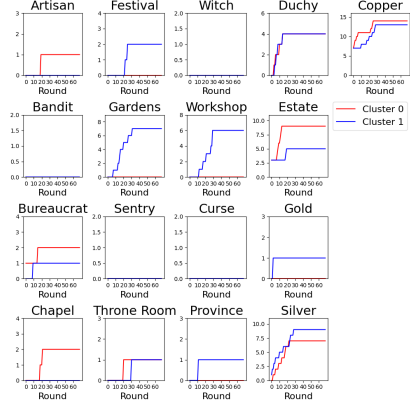


Fig. 12. DBSCAN cluster centroids for cardcount playtraces with $L_{0.5}$ -norm and MCTS restricted play with SD Kingdom card set.

play framework proposed in [1]. In this framework, the impact of a game dynamic or component is assessed by examining the impact of removing the access of a player to that component. In our case we prevent our MCTS agent from purchasing particular cards, which forces our agent to use alternative card combinations and strategies which are then highlighted via playtraces.

We use the same set-up as in section VI-B, with the exception that a single MCTS bot is restricted from buying Gardens, Workshop, Gold, Silver and Province cards. This prevents the bot from exploring the Gardens/Workshop and BMWG strategies already identified. Focusing on card count playtraces with an $L_{0.5}$ norm, we have the DBSCAN MDS plot shown in figure 11. The average silhouette score is 0.45, and shows two distinct clusters. Cluster one contains playtraces generated by the unrestricted MCTS bot, and cluster zero playtraces generated by the restricted MCTS bot. Examining the corresponding centroids in figure 12, we see that the restricted MCTS bot uses a combination of Artisan and Bureaucrat to obtain Silver cards, which are then used to purchase Estate cards for VPs. Artisan allows the player to gain a card of cost up to five and Bureaucrat directly adds a Silver card to the top of the player's draw deck. Whilst a sub-optimal strategy, it does highlight how a player can use Kingdom cards to increase their stock of treasure cards.

As an additional example, we perform a similar analysis with the FG1E Kingdom card set, restricting a single MCTS bot from using Smithy, Gold, Silver and Province cards. We focus on bi-grams and the MDS plot for DSCAN clustering is shown in figure 13. The two clusters have an average silhouette value of 0.42. Examining the centroids in figure 14, we see that

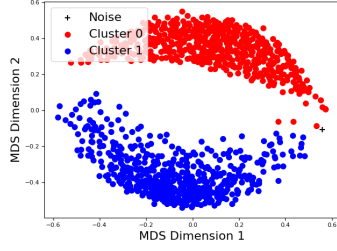


Fig. 13. MDS DBSCAN plot for bi-grams and MCTS restricted play with FG1E Kingdom card set.

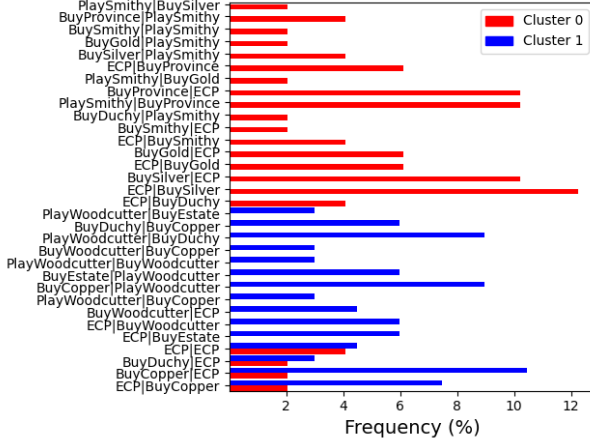


Fig. 14. DBSCAN bi-gram cluster centroids for MCTS restricted play with FG1E Kingdom card set.

cluster zero (corresponding to unrestricted MCTS playtraces) follows a Smithy plus Big Money strategy. However, for cluster one we see substantial use of the Woodcutter card. This card gives two treasure and plus one buy action, and can be an alternative source of treasure to Silver cards supporting the purchasing of the Duchy and Estate VP cards.

VII. CHALLENGES AND OPPORTUNITIES

A. Investigating game phase dependent strategies

Here we have examined playtraces for full games, however it would also be useful to examine how strategy and card usage changes between early, mid and late game phases. For example, in most DBCGs the early game is characterised by building an engine and the later game phase by the acquisition of victory points. Playtrace clustering on game phases could help highlight how play changes as a game progresses.

B. Impact of AI agent on strategy identification

One potential avenue for future research is the impact the choice of AI agent can have on our clustering results and hence strategy identification. Strategies can only be discovered if the AI agent plays those strategies. As a simple example, if we move from a score based heuristic to a simple win only heuristic for our MCTS playout policy, we see more use of the trashing mechanic and also use of Witch cards. Another possibility is to compare playtraces generated by MCTS with other AI algorithms, such as deep reinforcement learning based approaches [31].

C. Investigation of other Kingdom card set choices

We saw distinct clustering in the SD Kingdom card set, however both MCTS and human generated playtraces did not show distinct clustering for the FG1E set. This could be for a number of different reasons. Firstly, the SD Kingdom card set tends to have more distinct strategies, e.g. BMWG versus DW, whereas the FG1E Kingdom card set tends to focus around the use of Smithy and supporting cards. Additionally the varying skill level of human players and relatively small number of player logs may also be impacting the FG1E clustering. It would be interesting to observe how clustering changes as game mechanics change between Kingdom card sets, and as player skill level changes.

D. Application to other DBCGs and tabletop games

It would also be useful to see how these methods translate and what results are seen in other DBCGs. For example, the game *Ascension* is another archetypal DBCG, however it differs from *Dominion* in the existence of factions. These effectively act like suits in the game, with the total card set being split across four factions (plus some non-factions cards). Cards within factions are expected to have greater levels of synergies, which we would expect to see reflected in playtraces via strategies and clusters that focus on particular factions and combinations of factions.

Taking this one step further, the use of N-Grams based playtraces generated from player actions is applicable to any game where each player takes a discrete set of actions. Therefore clustering of this type of playtrace could potentially be used to explore the strategy space of a much broader class of tabletop games than DBCGs.

VIII. CONCLUSIONS

This paper defines two different types of playtraces, card count and N-Gram based playtraces, that when used in conjunction with AI based simulation of game sessions or human play can help provide insights into the strategy space of a DBCG. It was shown that using MCTS to simulate game sessions of *Dominion*, we can generate playtraces that when clustered can highlight different aspects of the space of possible strategies.

A number of insights can be drawn from examining playtraces and clusters for DBCGs (and potentially in tabletop games more broadly). Playtraces can help highlight weaker cards, in particular the infrequent use of a card may be a signal of a redundant mechanic or weak card. An example is the Workshop card, which along with the Woodcutter saw very little use during both (unrestricted) MCTS and human play, see figure 9.

Playtraces can also highlight particular card synergies, where we see cards played frequently in particular combinations. A good example, is the Gardens/Workshop combination, that was evident in playtraces generated by our MCTS agent. Another highlighted tactic was using Smithy to draw treasure cards to buy Gold and Province cards.

We were also able to see distinct clustering of playtraces into strategy groupings, particularly in the Size Distortion

Kingdom card set case. In both the card count and N-Gram playtrace cases, the BMWG and DW strategies from FSM agents were grouped into distinct and separated clusters. Additionally for the MCTS case, two clusters were observed, corresponding to a Gardens/Workshop strategy and also a simple Big Money with Gardens strategy. For FG1E, clustering was not observed and play tended to focus around a dominant strategy of building a Smithy based engine.

Finally, when drawing conclusions from our playtraces, we must consider the strengths and weakness of our AI agent, and use of restricted play can help in exploring the space of strategies available in a game.

Whilst we have explored only the one example of a DBCG here, the author expects that the approach described could support analysis of available strategies in other DBCGs and other genres of tabletop games, particularly when using the game agnostic N-Gram based playtrace approach.

ACKNOWLEDGMENTS

This paper is an abridged version of a masters dissertation and I would firstly like to thank my supervisor Raghubir Singh for valuable comments. In addition I would like to thank the following people from the Queen Mary University of London Game AI group. Simon Lucas for suggesting the use of N-Grams, James Goodman for his TAG implementation of *Dominion* and general helpful feedback, and finally Raluca Gaina and Diego Perez-Liebana for useful suggestions and supporting my use of TAG. Finally, I would like to thank Doug Zongker for providing archived human player log files from *Dominion.isotropic.org*.

REFERENCES

- [1] A. Jaffe, "Understanding game balance with quantitative methods," PhD thesis, University of Washington, UW Computer Science & Engineering, University of Washington, 1410 NE Campus Pkwy, Seattle, WA 98195, United States, July 2013, available at <https://digital.lib.washington.edu/researchworks/handle/1773/22797>.
- [2] D. Vaccarino, *Dominion*. Rio Grande Games, 2008.
- [3] J. Gary, R. Dougherty, and B. Kibler, *Ascension*. Stone Blade Entertainment, 2010.
- [4] R. Dougherty and D. Kastle, *Star Realms*. Wise Wizard Games, 2014.
- [5] C. Grace, *Star Wars: The Deck Building Game*. Fantasy Flight Games, 2023.
- [6] B. Ingram, B. Rosman, C. v. Alten, and R. Klein, "Play-style identification through deep unsupervised clustering of trajectories," in *2022 IEEE Conference on Games (CoG)*, 2022, pp. 393–400.
- [7] Y.-E. Liu, E. Andersen, R. Snider, S. Cooper, and Z. Popovic, "Feature-based projections for effective playtrace analysis," in *International Conference on Foundations of Digital Games*, 2011. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14029159>
- [8] J. Campbell, J. Tremblay, and C. Verbrugge, "Clustering player paths," in *International Conference on Foundations of Digital Games*, 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:18136549>
- [9] J. C. Osborn, B. Samuel, and M. Mateas, "Visualizing the strategic landscape of arbitrary games," *Information Visualization*, vol. 17, no. 3, pp. 196–217, 2018. [Online]. Available: <https://doi.org/10.1177/1473871617718377>
- [10] H. Bendekgey, "Clustering player strategies from variable-length game logs in dominion," *CoRR*, vol. abs/1811.11273, 2018. [Online]. Available: <http://arxiv.org/abs/1811.11273>
- [11] T. Mahlmann, J. Togelius, and G. Yannakakis, "Evolving card sets towards balancing dominion," in *2012 IEEE Congress on Evolutionary Computation, CEC 2012*, 06 2012, pp. 1–8.
- [12] C. Ford and M. Ohata, "Game balancing in dominion: An approach to identifying problematic game elements," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 11, pp. 12744–12750, Jun. 2022. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/21552>
- [13] N. Budijono, P. Goldman, J. Maloney, J. B. Mueller, P. Walker, J. Ladwig, and R. G. Freedman, "Ludus: An optimization framework to balance auto battler cards," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 11, pp. 12727–12734, Jun. 2022. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/21550>
- [14] R. D. Gaina, M. Balla, A. Dockhorn, R. Montoliu, and D. Perez-Liebana, "TAG: A Tabletop Games Framework," in *Experimental AI in Games (EXAG), AIIDE 2020 Workshop*, 2020.
- [15] M. Balla, G. E. M. Long, D. Jeurissen, J. Goodman, R. D. Gaina, and D. Perez-Liebana, "Pytag: Challenges and opportunities for reinforcement learning in tabletop games," 2023.
- [16] J. Goodman, D. Perez-Liebana, and S. Lucas, "Fingerprinting tabletop games," in *2021 IEEE Conference on Games (CoG)*, 2021, pp. 1–4.
- [17] J. Goodman, D. Perez Liebana, and S. Lucas, "Visualising multiplayer game spaces," 02 2022.
- [18] J. MacQueen *et al.*, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, vol. 1. Oakland, CA, USA, 1967, pp. 281–297.
- [19] C. Aggarwal, A. Hinneburg, and D. Keim, "On the surprising behavior of distance metric in high-dimensional space," *First publ. in: Database theory, ICDT 200, 8th International Conference, London, UK, January 4 - 6, 2001 / Jan Van den Bussche ... (eds.)*. Berlin: Springer; 2001, pp. 420–434 (=Lecture notes in computer science ; 1973), 02 2002.
- [20] L. Kaufman and P. J. Rousseeuw, *Partitioning Around Medoids (Program PAM)*. John Wiley & Sons, Inc., 2008, pp. 68–125. [Online]. Available: <http://dx.doi.org/10.1002/9780470316801.ch2>
- [21] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Knowledge Discovery and Data Mining*, 1996. [Online]. Available: <https://api.semanticscholar.org/CorpusID:355163>
- [22] A. Mead, "Review of the development of multidimensional scaling methods," *Journal of the Royal Statistical Society. Series D (The Statistician)*, vol. 41, no. 1, pp. 27–39, 1992. [Online]. Available: <http://www.jstor.org/stable/2348634>
- [23] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [24] P. I. Cowling, C. D. Ward, and E. J. Powley, "Ensemble determinization in monte carlo tree search for the imperfect information card game magic: The gathering," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 4, pp. 241–257, 2012.
- [25] J. Goodman, D. Perez-Liebana, and S. Lucas, "Multitree mcts in tabletop games," in *2022 IEEE Conference on Games (CoG)*, 2022, pp. 292–299.
- [26] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, vol. 47, no. 2–3, p. 235–256, may 2002. [Online]. Available: <https://doi.org/10.1023/A:1013689704352>
- [27] H. Finnsson and Y. Björnsson, "Simulation-based approach to general game playing," in *Aaai*, vol. 8, 2008, pp. 259–264.
- [28] J. Goodman, D. Perez Liebana, and S. Lucas, "Following the leader in multiplayer tabletop games," 04 2023, pp. 1–11.
- [29] D. Hiemstra, *N-Gram Models*. Boston, MA: Springer US, 2009, pp. 1910–1910. [Online]. Available: https://doi.org/10.1007/978-0-387-39940-9_935
- [30] P. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, no. 1, pp. 53–65, Nov. 1987. [Online]. Available: http://svn.donarmstrong.com/don/trunk/projects/research/papers_to_read/statistics/silhouettes_a_graphical_aid_to_the_interpretation_and_validation_of_cluster_analysis_rousseeuw_j_comp_app_math_20_53_1987.pdf
- [31] Yang, H. and Kuo, Y., "An ai for dominion using deep reinforcement learning," http://cs230.stanford.edu/projects/_fall_2019/reports/26260348.pdf, 2019, [Online; accessed 12-July-2023].